🎯 Express.js & MongoDB - Simplified Guide

For quick understanding with real-world analogies

PART 1: EXPRESS.JS EXPLAINED SIMPLY

What is Express.js?

Think of Express like a **restaurant**:

- **Your server** = The restaurant
- **Routes** = The menu items (what customers can order)
- **Middleware** = Kitchen staff who handle orders before serving
- **Request (req)** = Customer's order
- **Response (res)** = Food served to customer

```
const express = require('express');
const app = express();  // Create your restaurant

app.listen(3000);       // Open for business on port 3000
```

Understanding Routes (The Menu)

Routes tell your server: "When someone asks for THIS, give them THAT"

HTTP Methods = Types of Actions

Method	What it means	Restaurant Analogy
GET	"Give me data"	Customer reads the menu
POST	"Create something new"	Customer places new order
PUT	"Replace completely"	Customer changes entire order
PATCH	"Update partially"	Customer modifies one item
DELETE	"Remove this"	Customer cancels order

Basic Route Syntax

```
app.METHOD('/path', handlerFunction);

// Examples:
app.get('/users', (req, res) => {
    res.json({ message: 'Get all users' });
});

app.post('/users', (req, res) => {
```

```
    res.json({ message: 'Create a user' });
});
```

Understanding req (Request) - What the Customer Wants

The `req` object contains EVERYTHING the client sent to you.

3 Main Ways to Receive Data:

```
// 1. req.params - From the URL path itself
// URL: /users/123
app.get('/users/:id', (req, res) => {
    console.log(req.params.id);  // "123"
});

// 2. req.query - From ?key=value in URL
// URL: /users?page=2&limit=10
app.get('/users', (req, res) => {
    console.log(req.query.page);   // "2"
    console.log(req.query.limit);  // "10"
});

// 3. req.body - From POST/PUT data (needs middleware!)
// Sent in request body: { "name": "John", "email": "john@test.com" }
app.use(express.json());  // MUST add this to read JSON body!

app.post('/users', (req, res) => {
    console.log(req.body.name);   // "John"
    console.log(req.body.email);  // "john@test.com"
});
```

Visual:

```
URL: http://localhost:3000/users/123?page=2
                          ↑        ↑
                     req.params  req.query
                     (id=123)    (page=2)

Request Body (POST): { "name": "John" }
                          ↑
                     req.body
```

Understanding res (Response) - What You Send Back

The `res` object is HOW you respond to the client.

```
// Send JSON (most common for APIs)
res.json({ name: 'John', age: 30 });
```

```
// Send with specific status code
res.status(201).json({ message: 'Created!' });

// Send plain text
res.send('Hello World');

// Redirect to another page
res.redirect('/login');

// Common status codes:
// 200 = OK, success
// 201 = Created something new
// 400 = Bad request (client's fault)
// 401 = Not logged in
// 404 = Not found
// 500 = Server error (your fault)
```

Understanding Middleware (The Magic in Between)

Middleware = Functions that run BETWEEN receiving request and sending response

Think of it like an assembly line:

```
Request → [Middleware 1] → [Middleware 2] → [Route Handler] → Response
```

Middleware Structure:

```
function myMiddleware(req, res, next) {
    // Do something...
    console.log('Middleware ran!');

    next();  // MUST call next() to continue to next step!
}
```

Common Middleware Uses:

```
// 1. Logging - See every request
app.use((req, res, next) => {
    console.log(`${req.method} ${req.url}`);
    next();
});

// 2. Parse JSON body (built-in)
app.use(express.json());

// 3. Authentication check
const authenticate = (req, res, next) => {
    const token = req.headers.authorization;
```

```
    if (!token) {
        return res.status(401).json({ error: 'No token!' });
        // Notice: NO next() - we're stopping here
    }

    // Token exists, continue
    next();
};

// Use auth middleware on specific routes
app.get('/profile', authenticate, (req, res) => {
    res.json({ user: 'John' });
});
```

Middleware Order Matters!

```
// This runs for ALL routes (placed at top)
app.use(express.json());

// This runs only for /api routes
app.use('/api', authenticate);

// This is the actual route
app.get('/api/users', (req, res) => { ... });

// ERROR handler - must be LAST and have 4 parameters
app.use((err, req, res, next) => {
    res.status(500).json({ error: err.message });
});
```

Complete Express Example with Comments

```
const express = require('express');
const app = express();

// Step 1: Add middleware to parse JSON
app.use(express.json());

// Step 2: Add logging middleware
app.use((req, res, next) => {
    console.log(`${new Date().toISOString()} - ${req.method} ${req.url}`);
    next();
});

// Step 3: In-memory "database"
let users = [
    { id: 1, name: 'John', email: 'john@test.com' },
    { id: 2, name: 'Jane', email: 'jane@test.com' }
```

```
];

// Step 4: Define routes

// GET all users
app.get('/api/users', (req, res) => {
    res.json(users);
});

// GET one user by ID
app.get('/api/users/:id', (req, res) => {
    const id = parseInt(req.params.id);  // params are strings!
    const user = users.find(u => u.id === id);

    if (!user) {
        return res.status(404).json({ error: 'User not found' });
    }

    res.json(user);
});

// POST - Create new user
app.post('/api/users', (req, res) => {
    const { name, email } = req.body;  // Get from request body

    // Validation
    if (!name || !email) {
        return res.status(400).json({ error: 'Name and email required' });
    }

    const newUser = {
        id: users.length + 1,
        name,
        email
    };

    users.push(newUser);
    res.status(201).json(newUser);  // 201 = Created
});

// PUT - Update entire user
app.put('/api/users/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = users.findIndex(u => u.id === id);

    if (index === -1) {
        return res.status(404).json({ error: 'User not found' });
    }

    const { name, email } = req.body;
    users[index] = { id, name, email };  // Replace completely
```

```
    res.json(users[index]);
});

// DELETE - Remove user
app.delete('/api/users/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = users.findIndex(u => u.id === id);

    if (index === -1) {
        return res.status(404).json({ error: 'User not found' });
    }

    users.splice(index, 1);
    res.status(204).send();  // 204 = No Content (success, nothing to return)
});

// Step 5: Error handler (must be last!)
app.use((err, req, res, next) => {
    console.error(err);
    res.status(500).json({ error: 'Something went wrong!' });
});

// Step 6: Start server
app.listen(3000, () => console.log('Server running on port 3000'));
```

PART 2: MONGODB & MONGOOSE EXPLAINED SIMPLY

What is MongoDB?

Think of MongoDB like a **filing cabinet**:

- **Database** = The entire cabinet
- **Collection** = A drawer in the cabinet
- **Document** = A file/paper in the drawer
- **Field** = A piece of info on the paper

Unlike SQL (fixed spreadsheet), MongoDB is flexible (each paper can have different info).

MongoDB vs SQL Visual:

```
SQL Table (fixed columns):
┌────┬──────┬─────┬──────────────┐
│ id │ name │ age │ email        │
├────┼──────┼─────┼──────────────┤
│ 1  │ John │ 30  │ john@...     │
│ 2  │ Jane │ 25  │ jane@...     │
└────┴──────┴─────┴──────────────┘
```

```
MongoDB Collection (flexible documents):
{
  _id: ObjectId("..."),
  name: "John",
  age: 30,
  address: { city: "NYC", zip: "10001" }  // Nested!
}
{
  _id: ObjectId("..."),
  name: "Jane",
  hobbies: ["reading", "coding"]  // Array!
  // No age field - that's OK!
}
```

What is Mongoose?

Mongoose is a **translator** between your JavaScript code and MongoDB.

Without Mongoose: You talk to MongoDB in its raw language With Mongoose: You use nice JavaScript methods

---

Step 1: Connect to MongoDB

```
const mongoose = require('mongoose');

// Connection string format:
// mongodb://localhost:27017/databaseName
// Or for cloud (MongoDB Atlas):
// mongodb+srv://user:password@cluster.mongodb.net/databaseName

mongoose.connect('mongodb://localhost:27017/myapp')
    .then(() => console.log('Connected to MongoDB!'))
    .catch(err => console.error('Connection failed:', err));
```

Step 2: Create a Schema (Blueprint)

A Schema defines what your documents SHOULD look like.

```
const userSchema = new mongoose.Schema({
    // Basic field
    name: String,

    // With validation
    email: {
        type: String,
        required: true,      // Must have this field
        unique: true,        // No duplicates
        lowercase: true      // Convert to lowercase
    },
```

```
    // Number with constraints
    age: {
        type: Number,
        min: 0,
        max: 150
    },

    // Field with default value
    role: {
        type: String,
        enum: ['user', 'admin'],  // Only these values allowed
        default: 'user'
    },

    // Boolean with default
    isActive: {
        type: Boolean,
        default: true
    },

    // Nested object
    address: {
        city: String,
        country: String
    },

    // Array of strings
    hobbies: [String],

    // Reference to another collection (like foreign key)
    posts: [{
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Post'  // Name of the other model
    }]

}, { timestamps: true });  // Adds createdAt & updatedAt automatically
```

Step 3: Create a Model

A Model is what you actually use to interact with the database.

```
const User = mongoose.model('User', userSchema);
// 'User' → MongoDB will create collection called 'users' (lowercase, plural)

module.exports = User;
```

Step 4: CRUD Operations

CREATE - Add Documents

```
// Method 1: Create and save
const user = new User({
    name: 'John',
    email: 'john@test.com',
    age: 30
});
await user.save();

// Method 2: Create directly (shorthand)
const user = await User.create({
    name: 'Jane',
    email: 'jane@test.com'
});

// Create many at once
await User.insertMany([
    { name: 'User1', email: 'u1@test.com' },
    { name: 'User2', email: 'u2@test.com' }
]);
```

READ - Find Documents

```
// Find ALL
const allUsers = await User.find();

// Find with conditions
const activeUsers = await User.find({ isActive: true });

// Find ONE (first match)
const user = await User.findOne({ email: 'john@test.com' });

// Find by ID
const user = await User.findById('507f1f77bcf86cd799439011');

// Select specific fields (projection)
const names = await User.find().select('name email');
// Or exclude fields
const users = await User.find().select('-password');

// Sorting
const users = await User.find().sort({ createdAt: -1 });  // -1 = descending
const users = await User.find().sort('name');             // ascending

// Pagination
const page = 2;
const limit = 10;
const users = await User.find()
    .skip((page - 1) * limit)  // Skip first 10
```

```
    .limit(limit);              // Get next 10

// Count documents
const count = await User.countDocuments({ isActive: true });
```

UPDATE - Modify Documents

```
// Find and update (returns updated document)
const user = await User.findByIdAndUpdate(
    '507f1f77bcf86cd799439011',   // ID to find
    { name: 'New Name', age: 31 }, // What to update
    { new: true, runValidators: true }  // Options
);
// new: true → return the updated document (not old one)
// runValidators: true → check schema rules

// Update one (doesn't return the document)
await User.updateOne(
    { email: 'john@test.com' },    // Find condition
    { $set: { name: 'Johnny' } }   // Update
);

// Update many
await User.updateMany(
    { isActive: false },           // Find all inactive
    { $set: { role: 'archived' } } // Update their role
);
```

DELETE - Remove Documents

```
// Find and delete (returns deleted document)
const deleted = await User.findByIdAndDelete('507f1f77bcf86cd799439011');

// Delete one
await User.deleteOne({ email: 'john@test.com' });

// Delete many
await User.deleteMany({ isActive: false });
```

Query Operators - The $ Symbols

These let you make complex queries.

Comparison Operators

```
// Greater than $gt, greater or equal $gte
await User.find({ age: { $gt: 18 } });     // age > 18
await User.find({ age: { $gte: 18 } });    // age >= 18
```

```
// Less than $lt, less or equal $lte
await User.find({ age: { $lt: 30 } });     // age < 30
await User.find({ age: { $lte: 30 } });    // age <= 30

// Not equal $ne
await User.find({ role: { $ne: 'admin' } }); // role NOT admin

// In array $in, not in $nin
await User.find({ role: { $in: ['admin', 'moderator'] } });
await User.find({ status: { $nin: ['banned', 'suspended'] } });

// Range
await User.find({ age: { $gte: 18, $lte: 65 } }); // Between 18-65
```

Logical Operators

```
// AND (implicit - just add conditions)
await User.find({ isActive: true, role: 'admin' });

// AND (explicit)
await User.find({
    $and: [
        { age: { $gte: 18 } },
        { age: { $lte: 30 } }
    ]
});

// OR
await User.find({
    $or: [
        { role: 'admin' },
        { role: 'moderator' }
    ]
});
```

Update Operators

```
// $set - Set field value
await User.updateOne({ _id: id }, { $set: { name: 'New Name' } });

// $unset - Remove a field
await User.updateOne({ _id: id }, { $unset: { tempField: 1 } });

// $inc - Increment number
await User.updateOne({ _id: id }, { $inc: { loginCount: 1 } });
await User.updateOne({ _id: id }, { $inc: { balance: -50 } }); // Decrement

// $push - Add to array
```

```
await User.updateOne({ _id: id }, { $push: { hobbies: 'gaming' } });

// $pull - Remove from array
await User.updateOne({ _id: id }, { $pull: { hobbies: 'gaming' } });

// $addToSet - Add to array only if doesn't exist
await User.updateOne({ _id: id }, { $addToSet: { hobbies: 'reading' } });
```

Population - MongoDB's "Join"

When you have references between collections, `populate()` fetches the actual data.

```
// Post schema has reference to User
const postSchema = new mongoose.Schema({
    title: String,
    content: String,
    author: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User'  // Reference to User model
    }
});

// WITHOUT populate - you get just the ID
const post = await Post.findById(postId);
console.log(post.author);  // "507f1f77bcf86cd799439011" (just ID)

// WITH populate - you get the full user object
const post = await Post.findById(postId).populate('author');
console.log(post.author);  // { _id: ..., name: 'John', email: '...' }

// Populate specific fields only
const post = await Post.findById(postId).populate('author', 'name email');
// post.author = { _id: ..., name: 'John', email: '...' }
```

Complete Express + MongoDB Example

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

// Middleware
app.use(express.json());

// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/myapp');

// Define Schema
const userSchema = new mongoose.Schema({
```

```
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    age: { type: Number, min: 0 }
}, { timestamps: true });

// Create Model
const User = mongoose.model('User', userSchema);

// Routes

// GET all users
app.get('/api/users', async (req, res) => {
    try {
        const users = await User.find();
        res.json(users);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// GET one user
app.get('/api/users/:id', async (req, res) => {
    try {
        const user = await User.findById(req.params.id);
        if (!user) {
            return res.status(404).json({ error: 'User not found' });
        }
        res.json(user);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// POST create user
app.post('/api/users', async (req, res) => {
    try {
        const user = await User.create(req.body);
        res.status(201).json(user);
    } catch (error) {
        res.status(400).json({ error: error.message });
    }
});

// PUT update user
app.put('/api/users/:id', async (req, res) => {
    try {
        const user = await User.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true, runValidators: true }
        );
        if (!user) {
```

```
            return res.status(404).json({ error: 'User not found' });
        }
        res.json(user);
    } catch (error) {
        res.status(400).json({ error: error.message });
    }
});

// DELETE user
app.delete('/api/users/:id', async (req, res) => {
    try {
        const user = await User.findByIdAndDelete(req.params.id);
        if (!user) {
            return res.status(404).json({ error: 'User not found' });
        }
        res.status(204).send();
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});

// Start server
app.listen(3000, () => console.log('Server running'));
```

Quick Reference Card

Express

```
app.use(middleware)        → Apply middleware
app.get/post/put/delete    → Define routes
req.params                 → URL params (/users/:id)
req.query                  → Query string (?page=1)
req.body                   → POST/PUT body data
res.json()                 → Send JSON response
res.status(code)           → Set status code
next()                     → Continue to next middleware
```

MongoDB/Mongoose

```
Model.create()             → Insert new
Model.find()               → Get all matching
Model.findById()           → Get by ID
Model.findOne()            → Get first match
Model.findByIdAndUpdate()  → Update by ID
Model.findByIdAndDelete()  → Delete by ID
.select('field1 field2')   → Choose fields
.sort('-field')            → Sort (- = descending)
.limit(10)                 → Limit results
```

```
.skip(10)                  → Skip results
.populate('ref')           → Join data
```

You've got this! The more you practice, the clearer it becomes. 🚀